INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Plano interactivo de un ambiente con Canvas en Jetpack Compose				
NÚMERO DE LA PRÁCTICA:	07	**AÑO LECTIVO:**	2026	**NRO. SEMESTRE:**	B
FECHA DE PRESENTACIÓN:	13/10/2026	**HORA DE PRESENTACIÓN:**	11:59:00		
INTEGRANTE(s): - Hancco Mullisaca, Sergio Danilo - Huacani Jara, Denise Andrea				**NOTA (0 - 20):**	Nota colocada por el docente
DOCENTE: Roxana Evelyn Limache Calatayud					

SOLUCIÓN Y RESULTADOS

I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS

Ejercicio 1: Plano Interactivo en Jetpack Compose

Enunciado

Desarrollar una aplicación interactiva en Jetpack Compose denominada **PlanoInteractivo_Compose**, en la cual se represente un plano arquitectónico simple con salones y marcadores interactivos dibujados mediante la API de gráficos de Compose (`Canvas`).

Desarrollo Paso a Paso

Paso 1: Creación del Proyecto

Se creó un nuevo proyecto en Android Studio seleccionando la plantilla **Empty Activity (Compose)**. Se configuró el nombre del proyecto como `PlanoInteractivo_Compose`, el paquete base `com.example.planointeractivo_compose`, lenguaje Kotlin y el SDK mínimo compatible con API 34 o superior.

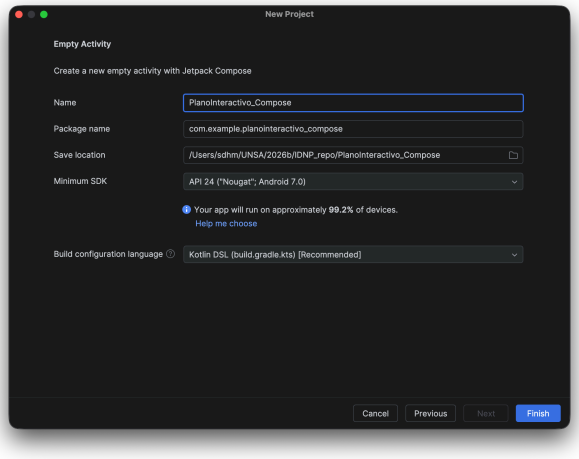


Figura 1: Configuración inicial del nuevo proyecto en Android Studio.

Paso 2: Configuración de Dependencias e Importaciones

En el archivo `MainActivity.kt`, debajo de la definición del paquete, se incorporaron las importaciones necesarias para el soporte de gráficos en Compose (`Canvas`, `Offset`, `Size`, `Color`), manejo de eventos táctiles (`pointerInput`, `detectTapGestures`), administración de estado reactivo (`remember`, `mutableStateOf`, `getValue`, `setValue`) y componentes visuales de Material 3 (`AlertDialog`, `Surface`, `Text`, `TextButton`).

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.planointeractivo_compose.ui.theme.PlanoInteractivo_ComposeTheme
```

Figura 2: Declaración de importaciones requeridas en MainActivity.kt.

Paso 3: Definición del Modelo de Datos (`Marcador`)

Fuera de cualquier función composable, se definió la clase de datos (`data class`) `Marcador`. Esta estructura encapsula las propiedades geométricas y descriptivas de cada punto de interés: las coordenadas en el plano (`offset: Offset`), el radio de cobertura táctil (`radio: Float`), así como el título y la descripción textual del ambiente.

```
data class Marcador(
    val offset: Offset,
    val radio: Float,
    val titulo: String,
    val descripcion: String
)
```

Figura 3: Definición de la data class Marcador.

Paso 4: Inicialización del Composable `PlanoAmbiente` y Gestión de Estado

Se implementó la función composable `PlanoAmbiente`, inicializando una lista inmutable de marcadores de muestra mediante `remember` (Sala A y Sala B con sus respectivas coordenadas cartesianas y radios de 40f). Asimismo, se declaró la variable reactiva `marcadorSeleccionado` delegada por `mutableStateOf<Marcador?>(null)` para controlar la selección activa y la visibilidad del diálogo.

```
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset( x = 100f,  y = 150f),  radio = 40f,  titulo = "Sala A",  descripcion = "Sala de cómputo 1"),
            Marcador(Offset( x = 250f,  y = 150f),  radio = 40f,  titulo = "Sala B",  descripcion = "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>( value = null) }
// el Canvas se agrega en el Paso 5
}
```

Figura 4: Declaración del Composable PlanoAmbiente y variables de estado iniciales.

Paso 5: Declaración del Lienzo `Canvas` y Reconocedor de Gestos

Se declaró el componente `Canvas` con un modificador que ocupa todo el ancho disponible (`fillMaxWidth()`) y una altura fija de `300.dp`. Se vinculó el detector de gestos táctiles mediante `Modifier.pointerInput(Unit)` y la función `detectTapGestures`, preparando el bloque de captura de toques antes de definir el renderizado gráfico.

```
    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput( key1 = Unit) {
                detectTapGestures { offset ->
                // aquí se calcula el marcador tocado, en el Paso 8
                }
            }
    ) {
    // aquí se dibuja el plano, en los Pasos 6 y 7
    }
```

Figura 5: Configuración del Canvas con modificador pointerInput y detectTapGestures.

Paso 6: Renderizado de la Estructura Arquitectónica (Salones y Pared)

Dentro del bloque de dibujo del `Canvas`, se renderizaron las áreas estructurales del plano. Utilizando `drawRect`, se dibujaron el **Salón 1** (color azul claro `0xFFBBDEFB`) y el **Salón 2** (color verde claro `0xFFC8E6C9`) a partir de sus desplazamientos iniciales y dimensiones. Posteriormente, se trazó la pared divisoria vertical con `drawLine` en color gris oscuro (`Color.DarkGray`) con un grosor de trazo de `4f`.

```
Canvas(
    modifier = Modifier
        .fillMaxWidth()
        .height(300.dp)
        .pointerInput( key1 = Unit) {
            detectTapGestures { offset ->
                // Salón 1
                drawRect(
                    color = Color(0xFFBBDEFB),
                    topLeft = Offset( x = 20f,  y = 20f),
                    size = Size( width = 150f,  height = 250f)
                )
                // Salón 2
                drawRect(
                    color = Color(0xFFC8E6C9),
                    topLeft = Offset( x = 180f,  y = 20f),
                    size = Size( width = 150f,  height = 250f)
                )
                // Pared divisoria
                drawLine(
                    color = Color.DarkGray,
                    start = Offset( x = 170f,  y = 20f),
                    end = Offset( x = 170f,  y = 270f),
                    strokeWidth = 4f
                )
                // aquí se dibujan los marcadores, en el Paso 7
            }
        }
```

Figura 6: Renderizado geométrico de los salones y el muro divisorio con drawRect y drawLine.

Paso 7: Renderizado de Marcadores Interactivos

Se iteró sobre la colección `marcadores` dentro del lienzo utilizando `drawCircle`. Cada marcador se dibujó en color rojo en la posición correspondiente (`marcador.offset`) y con el radio predeterminado (`marcador.radio`), completando el renderizado visual de los puntos de interés sobre el plano arquitectónico.

```
        strokeWidth = 4f
    )
    // Pared divisoria
    drawLine(
        color = Color.DarkGray,
        start = Offset( x = 170f,  y = 20f),
        end = Offset( x = 170f,  y = 270f),
        strokeWidth = 4f
    )
    marcadores.forEach { marcador ->
        drawCircle(
            color = Color.Red,
            radius = marcador.radio,
            center = marcador.offset
        )
    }
}
```

Figura 7: Dibujo de los marcadores circulares sobre el plano mediante drawCircle.

Paso 8: Detección de Colisiones Táctiles

En el manejador `detectTapGestures`, se implementó el cálculo euclidiano para verificar si la pulsación táctil intersecta con algún marcador. Al recibir la coordenada `offset` del toque, se evalúa si la distancia `(offset - marcador.offset).getDistance()` es menor o igual al radio del elemento. En caso afirmativo, se actualiza el estado `marcadorSeleccionado`.

```
Canvas(
    modifier = Modifier
        .fillMaxWidth()
        .height(300.dp)
        .pointerInput( key1 = Unit) {
            detectTapGestures { offset ->
                marcadores.forEach { marcador ->
                    if ((offset - marcador.offset).getDistance() <= marcador.radio) {
                        marcadorSeleccionado = marcador
                    }
                }
            }
        }
){
                // Salón 1
```

Figura 8: Lógica matemática de colisión y selección en detectTapGestures.

Paso 9: Integración del Diálogo de Información (`AlertDialog`)

Fuera del bloque `Canvas`, se utilizó la función de alcance `marcadorSeleccionado?.let { ... }` para condicionar el despliegue del componente `AlertDialog`. El diálogo muestra dinámicamente el título y la descripción del marcador pulsado, e incorpora un botón de confirmación («Cerrar») y la acción `onDismissRequest` que restablecen el estado a `null` para descartar la ventana modal.

```
2 Usages
@Composable
fun PlanoAmbiente() {

    // Lista de marcadores del plano
    val marcadores = remember {
        listOf(
            Marcador(
                Offset( x = 100f, y = 150f),
                radio = 40f,
                titulo = "Sala A",
                descripcion = "Sala de cómputo 1"
            ),
            Marcador(
                Offset( x = 250f, y = 150f),
                radio = 40f,
                titulo = "Sala B",
                descripcion = "Sala de cómputo 2"
            )
        )
    }

    // Estado del marcador seleccionado
    var marcadorSeleccionado by remember {
        mutableStateOf<Marcador?>( value = null)
    }

    // Lienzo interactivo
    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput( key1 = marcadores) {
                detectTapGestures { offset ->
```

Figura 9: Incorporación del diálogo modal AlertDialog para la visualización de detalles.

Paso 10: Integración en `MainActivity` y Vista Previa

En la clase `MainActivity`, dentro del método `onCreate` y el bloque `setContent`, se invocó `PlanoAmbiente` envuelto en el tema de la aplicación (`PlanoInteractivo_ComposeTheme`) y un contenedor `Surface`. Adicionalmente, se configuró una función de previsualización anotada con `@Preview` (`PlanoAmbientePreview`) para verificar el diseño en tiempo de desarrollo.

```
1 Usage
class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContent {
            PlanoInteractivo_ComposeTheme {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ) {
                    PlanoAmbiente()
                }
            }
        }
    }
}
```

Figura 10: Invocación del Composable en setContent de MainActivity y función Preview.

Paso 11: Ejecución y Validación en Emulador

Se ejecutó la aplicación en el emulador de Android (API 34). Se constató el renderizado fiel de los ambientes arquitectónicos y marcadores, así como la respuesta interactiva al pulsar sobre cada uno de ellos, abriendo correctamente los diálogos modales informativos de «Sala A» y «Sala B».

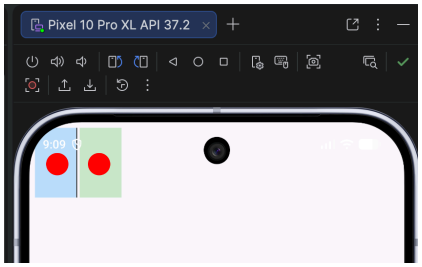


Figura 11: Visualización del plano y marcadores en el emulador.



Figura 12: Diálogo informativo modal: Sala A.



Figura 13: Diálogo informativo modal: Sala B.

Ejercicio 2

Enunciado

Instrucciones del segundo ejercicio.

Desarrollo

Solución desarrollada por el integrante encargado.

```python
# Solución del Ejercicio 2
def calcular_estadisticas(valores):
    promedio = sum(valores) / len(valores)
    varianza = sum((x - promedio) ** 2 for x in valores) / len(valores)
    return promedio, varianza

prom, var = calcular_estadisticas([10, 12, 23, 23, 16, 23, 21, 16])
print(f"Promedio: {prom:.2f}, Varianza: {var:.2f}")
```

Ejercicio 3

Enunciado

Instrucciones del tercer ejercicio.

Desarrollo

Solución desarrollada por el integrante encargado.

```python
# Solución del Ejercicio 3
def busqueda_binaria(arr, objetivo):
    izq, der = 0, len(arr) - 1
    while izq <= der:
        medio = (izq + der) // 2
        if arr[medio] == objetivo:
            return medio
        elif arr[medio] < objetivo:
            izq = medio + 1
```

```
        else:
            der = medio - 1
    return -1

lista = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]
print(f"Índice de 23: {busqueda_binaria(lista, 23)}")
```

II. SOLUCIÓN DEL CUESTIONARIO

Pregunta 1
Respuesta de la primera pregunta, redactada por el integrante encargado.

Pregunta 2
Respuesta de la segunda pregunta, redactada por el integrante encargado.

Pregunta 3
Respuesta de la tercera pregunta, redactada por el integrante encargado.

III. CONCLUSIONES

- Redacta aquí la primera conclusión de la práctica.
- Redacta aquí la segunda conclusión de la práctica.
- Redacta aquí la tercera conclusión de la práctica.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

[1] I. Sommerville, *Ingeniería del Software*, 9.ª ed. Madrid: Pearson Educación, 2011.

[2] R. S. Pressman, *Ingeniería del Software: Un enfoque práctico*, 7.ª ed. México: McGraw-Hill, 2010.

[3] Escuela Profesional de Ingeniería de Sistemas, «Guía de Prácticas de Laboratorio». 2024.